Study Tracker

Web App

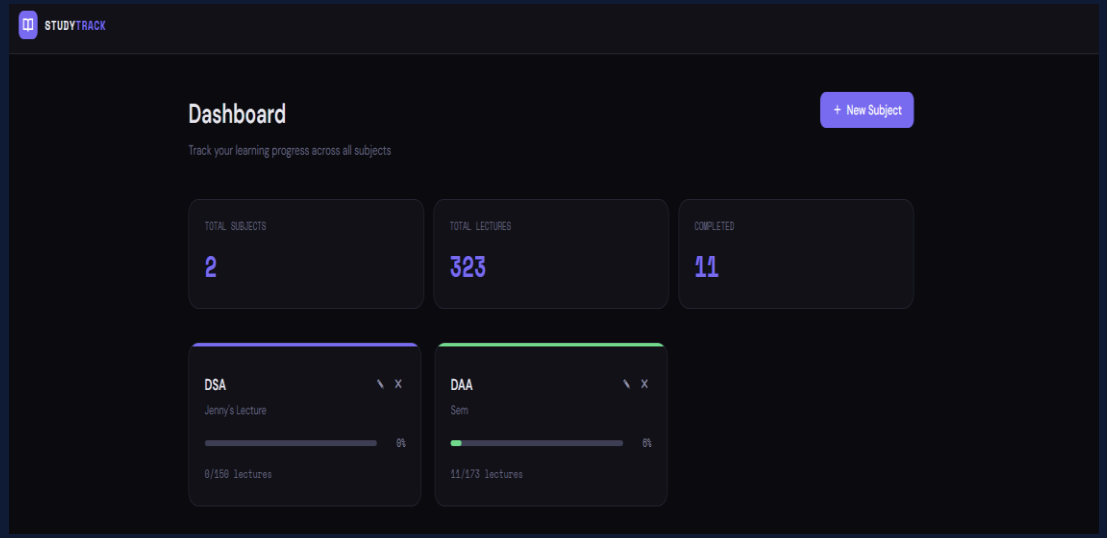
Track lectures. Measure progress. Study smarter.

Rajdeep Dey

Priyanshi Agarwal

Priyam Mondal

Rahul Das



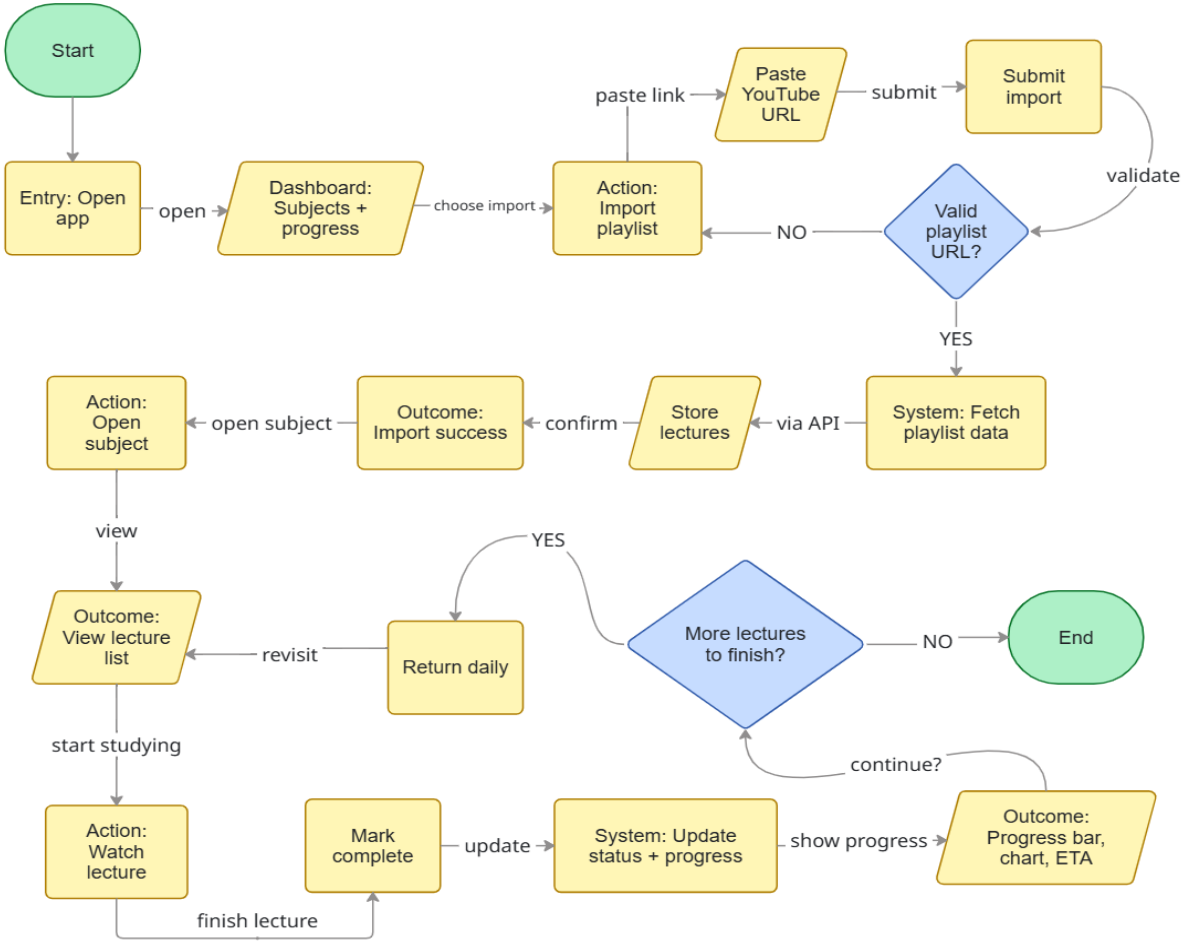
The Problem

Students lack a structured way to monitor lecture completion across multiple subjects - leading to missed topics and poor exam prep.

The Solution

A web app that imports YouTube playlists, tracks lecture progress, visualises completion stats, and predicts study deadlines.

User Journey & System Work



Features

- YouTube Playlist Integration
- Lecture Tracking System
- Progress Tracking
- Completion Prediction

TECH STACK

Frontend

React + Vite

Backend

Python FastAPI

Database

MySQL

API

YouTube Data API

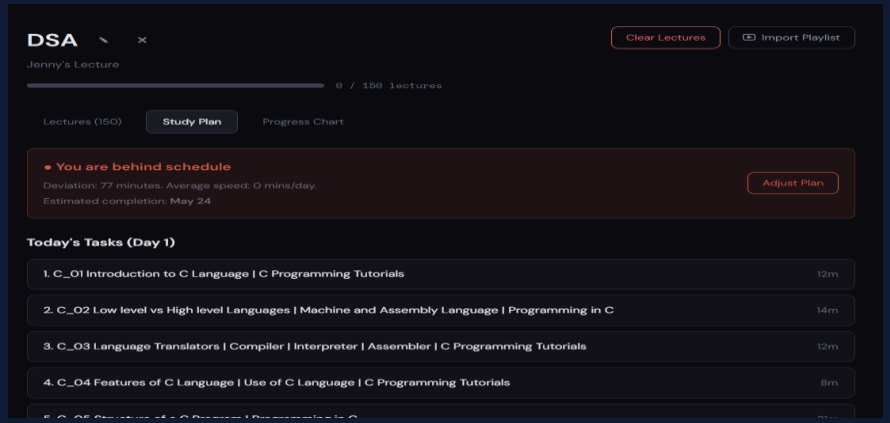
Use Cases & Future Roadmap

Real-Life Use Cases

- Students preparing for university exams or competitive tests (GATE, JEE, UPSC)
- Self-learners following YouTube courses on programming, design, or data science

Future Improvements

- AI Study Planner - auto-generate daily schedules based on deadlines & pace
- Streak System - gamified daily study streaks with badges and milestones
- Smart Scheduling - adaptive reminders based on weakest subjects & availability
- Google Calender – study schedule on calender



Smart Study Planner

Per day Lecture breakdown

B. P. Poddar Institute of Management & Technology

Academic Year – 2025-2026 Department: CSE

Course Name: DESIGN & ANALYSIS OF ALGORITHM

Course Code: PCC-CS494 Year: 2nd Semester: 4th

Report Project Title: Student Study-Tracker

Student Details:

STUDENT NAME	UNIVERSITY ROLL
RAJDEEP DEY	11500124125
RAHUL DAS	11500124127
PRIYAM MONDAL	11500124133
PRIYANSHI AGARWAL	11500124134

1. Abstract

The Study Tracker Web App is a full-stack web application designed to help students systematically track their lecture progress across multiple subjects. In the modern educational landscape, learners frequently rely on YouTube playlists as their primary source of instructional content. However, there is a significant lack of dedicated tools for monitoring completion, measuring progress, and predicting study timelines based on those playlists.

This application bridges that gap by providing a unified dashboard that integrates with the YouTube Data API to automatically import lecture playlists, tracks individual lecture completion through interactive checkboxes, auto-calculates subject-wise progress percentages, and visualizes completion trends over time. An estimated completion predictor further assists students in planning their study schedules effectively.

Built on a modern technology stack - React (Vite) for the frontend, Python FastAPI for the backend, and MySQL as the relational database - the system follows a clean layered architecture ensuring data integrity, scalability, and maintainability. This report covers the project's objectives, system design, feature implementation, use cases, and future roadmap.

> Key Highlights:
> - YouTube playlist auto-import via YouTube Data API
> - Real-time progress tracking with duplicate-safe updates
> - Subject-wise analytics dashboard and completion estimator
> - Scalable REST API architecture using Python FastAPI

2. Introduction

2.1 Background & Motivation

The explosion of online educational content on YouTube has transformed how students learn. Millions of learners follow structured playlists for subjects ranging from programming and mathematics to competitive exam preparation. Despite the abundance of content, students face a common challenge: there is no reliable way to track which lectures they have completed, how much of a course remains, or when they are likely to finish at their current pace.

Manual tracking through spreadsheets is error-prone and tedious. Existing Learning Management Systems (LMS) are designed for institutional use and do not integrate with YouTube playlists. This gap motivated the development of the Study Tracker Web App — a lightweight, student-centric tool that automates lecture tracking and delivers meaningful analytics.

2.2 Problem Statement

Students using YouTube playlists for self-study face the following core problems:

- No structured method to record and visualize lecture completion across multiple subjects simultaneously.
- Inability to accurately measure progress percentage or predict study completion timelines.
- Manual tracking leads to duplicate counting errors and inconsistent data.
- No unified interface that combines content import, tracking, and analytics in one place.

2.3 Objectives

The primary objectives of this project are:

1. To build a web application that imports YouTube playlists automatically using the YouTube Data API.
2. To enable students to mark individual lectures as completed with instant progress recalculation.
3. To provide a visual analytics dashboard with per-subject completion bars and time-series charts.
4. To implement an intelligent deadline estimator based on the learner's current study pace.
5. To ensure data integrity through server-side validation preventing duplicate progress updates.

3. Technology Stack

The Study Tracker Web App is built on a carefully chosen set of modern, open-source technologies that balance developer productivity, runtime performance, and scalability. The stack follows a clean separation between presentation, business logic, and data layers.

3.1 Stack Overview

Layer	Technology	Role	Key Benefit
Frontend	React 18 + Vite	Component-based SPA	Fast HMR, optimized builds
Backend	Python FastAPI	REST API server	Async support, auto docs
Database	MySQL 8	Relational store	ACID compliance, joins
External API	YouTube Data API v3	Playlist & video data	Official Google API
ORM	SQLAlchemy	Database abstraction	Query safety, migrations
Styling	Tailwind CSS	Utility-first CSS	Responsive, no bloat

3.2 Why This Stack?

React + Vite

React's component model is ideal for building reusable UI elements like progress bars, lecture checklists, and chart widgets. Vite replaces traditional Webpack bundlers with a lightning-fast native ES module dev server, dramatically improving developer experience during iteration.

Python FastAPI

FastAPI was selected over alternatives like Flask or Django REST Framework for its native support for Python type hints, automatic OpenAPI documentation generation, and asynchronous request handling. This makes the backend highly readable and easy to test, while being production-grade in performance.

MySQL

MySQL provides a robust relational model for storing subjects, playlists, and lecture completion records. Relational integrity constraints (foreign keys, unique constraints) are used to enforce the no-duplicate-update rule at the database level, which is a core data integrity requirement.

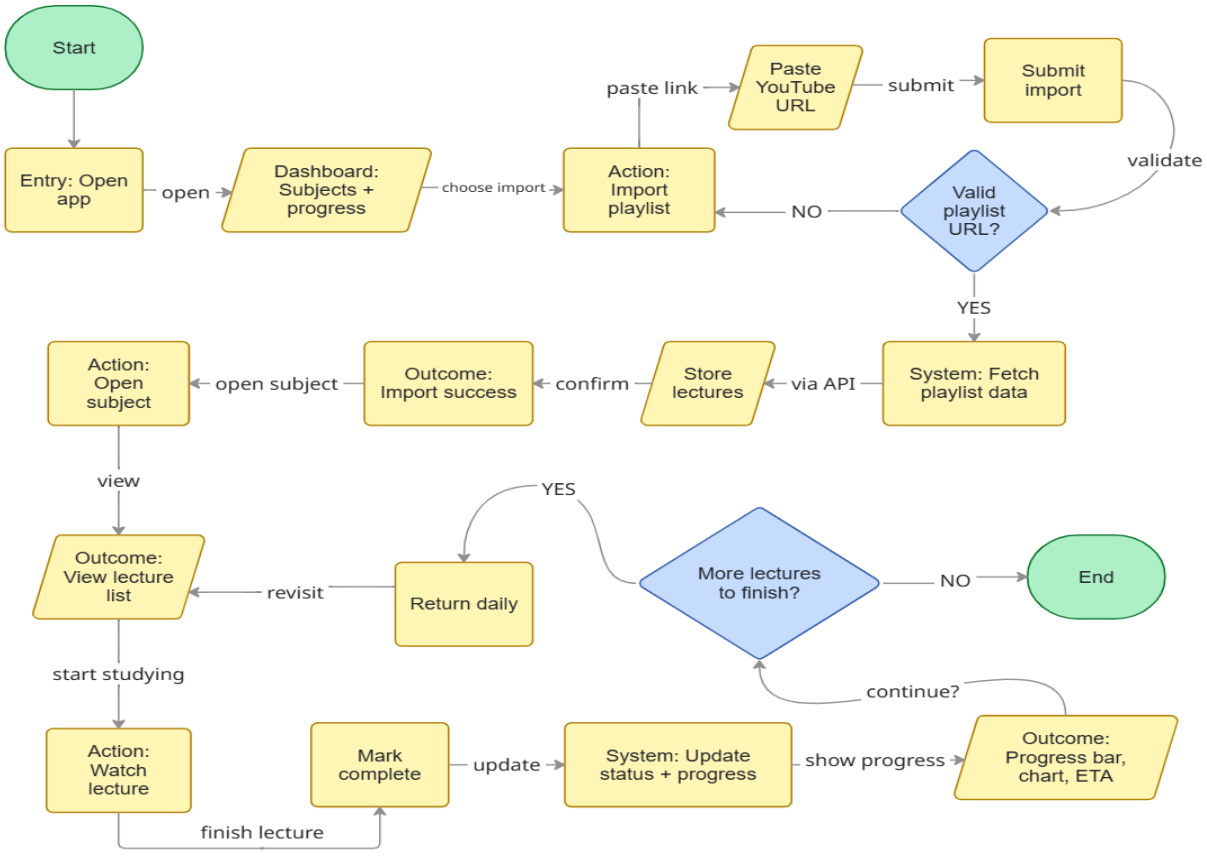
4. System Architecture

The application is structured using a layered architecture, separating concerns across four distinct tiers. Each layer communicates only with its immediate neighbors, ensuring loose coupling and high cohesion.

4.1 Data Flow

A typical user interaction follows this sequence:

- User pastes a YouTube playlist URL into the Import form on the React frontend.
- React sends a POST /api/playlist/import request to the FastAPI backend.
- FastAPI validates the request and passes it to PlaylistService.
- PlaylistService calls the YouTube Data API to fetch all video titles and IDs.
- Fetched lectures are stored in MySQL (subjects and lectures tables) with duplicate checks.
- Frontend receives the response and re-renders the subject dashboard.
- When a user marks a lecture complete, a PATCH /api/progress/{id} request updates the record.
- ProgressService validates the update (no duplicate), recalculates the percentage, and returns the new state.

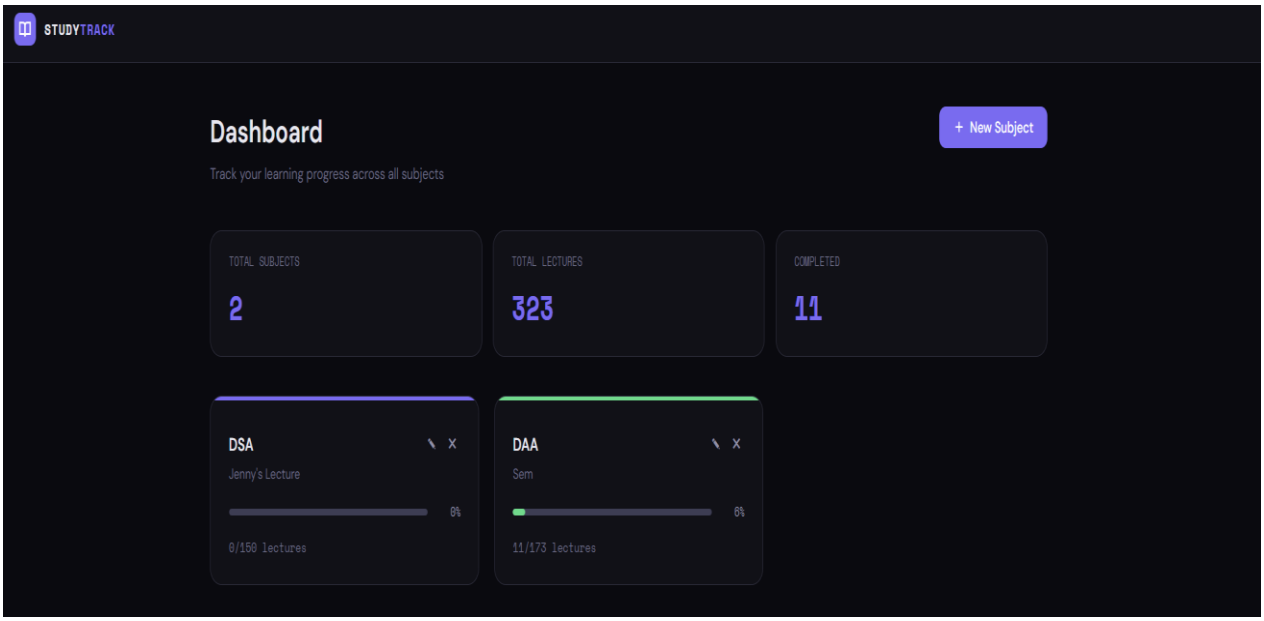


5. Features & Implementation

5.1 Dashboard with Subject-Wise Progress

The main dashboard provides a bird's-eye view of all subjects the student is tracking. Each subject card displays the playlist name, the total number of lectures, the number completed, and an animated progress bar indicating the completion percentage. Progress values are fetched via a dedicated GET /api/subjects endpoint that aggregates completion data server-side.

The dashboard is designed to be responsive, adapting to both desktop and mobile screen sizes using Tailwind CSS utility classes. Subject cards are rendered as React components with local state for optimistic UI updates — the checkbox state changes instantly in the UI while the backend call completes asynchronously.

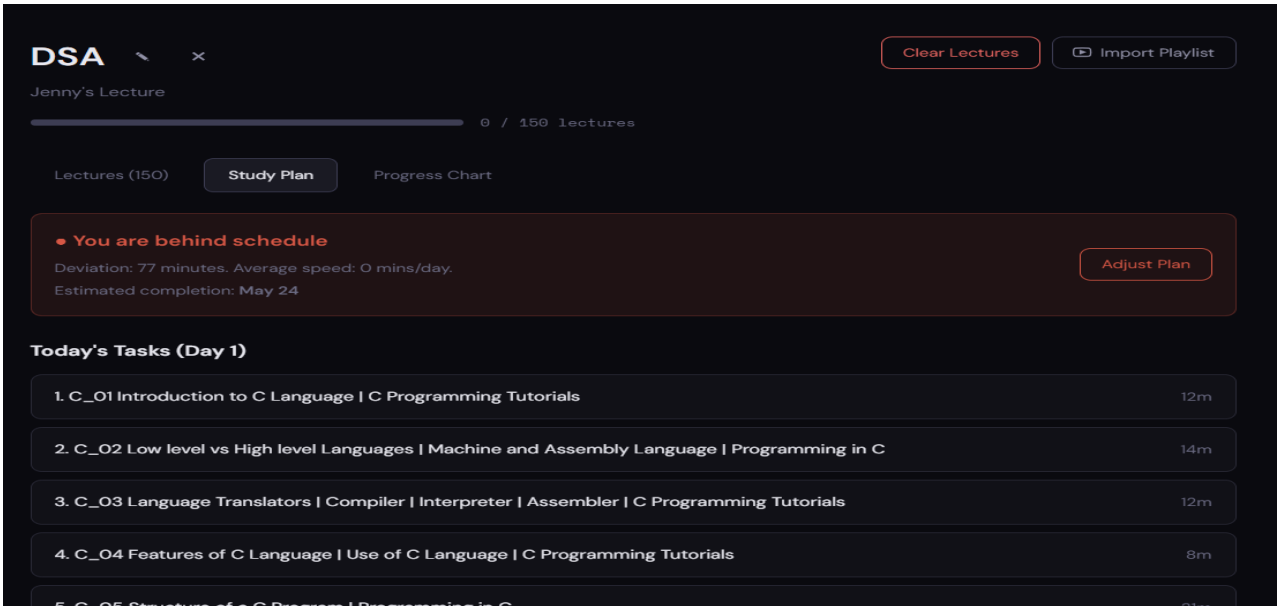


5.2 YouTube Playlist Import

The playlist import feature is the cornerstone of the application's automation capability. Students no longer need to manually enter lecture titles. The workflow is:

- Student enters the YouTube playlist URL and assigns a subject name.
- The backend extracts the playlist ID from the URL using regex.
- A request is made to the YouTube Data API v3 (playlistItems endpoint) with pagination support.
- All video metadata (title, videoId, position) is extracted and stored in the database.
- Duplicate imports are prevented at the database level using a UNIQUE constraint on (subject_id, video_id).

YouTube API Integration Detail:
Endpoint : GET https://www.googleapis.com/youtube/v3/playlistItems
Params : part=snippet, playlistId=<id>, maxResults=50, pageToken=<token>
Handles : Pagination via nextPageToken to fetch all videos regardless of playlist size
Rate : Stays within YouTube Data API v3 quota (10,000 units/day default)



5.3 Lecture Tracking with Checkbox Completion

Each lecture imported from a playlist is rendered as a row in the lecture list, accompanied by a checkbox. When a student checks a lecture as complete, the React component dispatches a PATCH request to the backend. The ProgressService validates that the completion has not been recorded before (idempotency check), then inserts or updates the progress record.

This design ensures data integrity even if the client sends duplicate requests due to network retries. The database uses a unique composite key on (user_id, lecture_id) to enforce this constraint at the storage layer.

5.4 Progress Percentage Auto-Calculation

Subject-level progress is calculated server-side using a simple but robust formula: Progress (%) = (Completed Lectures / Total Lectures) × 100. This calculation is performed inside a SQLAlchemy query that counts completed records per subject, avoiding the need for application-level loops. The result is returned as part of every subject listing response, keeping the frontend stateless with respect to progress.

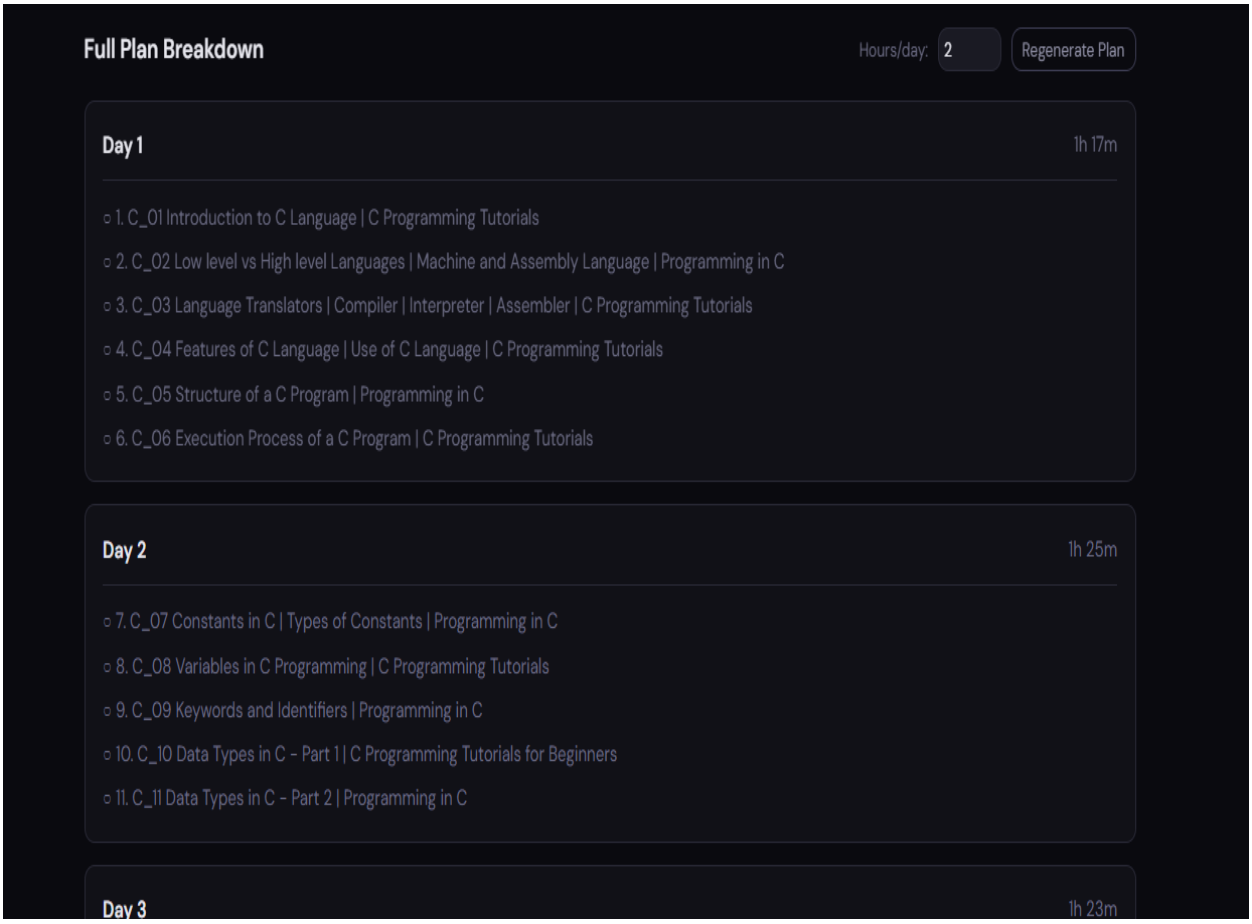
5.5 Progress Chart — Completion Over Time

A time-series chart visualizes how the student's completion has grown over calendar days. Each time a lecture is marked complete, a timestamp is recorded. The backend aggregates these timestamps per day to produce a cumulative completion series. The React frontend renders this data using a charting library, displaying an area chart that clearly shows the learning velocity.

5.6 Estimated Completion Prediction

The PredictionService calculates an estimated completion date using the student's recent study velocity. It computes the average number of lectures completed per day over the last seven days, then divides the remaining lecture count by this average to estimate how many more days are needed. The result is displayed prominently on the dashboard as an actionable insight.

> Prediction Formula:
> velocity = lectures_completed_last_7_days / 7
> remaining = total_lectures - completed_lectures
> days_to_done = remaining / velocity (if velocity > 0)
> est_date = today + timedelta(days=days_to_done)



6. Database Design

The MySQL database is organized around four core tables. Relationships are enforced via foreign keys, and unique constraints protect data integrity.

Table	Key Columns	Purpose	Constraint
subjects	id, name, user_id, created_at	Stores each subject/course	Primary entity
playlists	id, subject_id, yt_playlist_id, title	Links YT playlist to subject	One per subject
lectures	id, playlist_id, video_id, title, position	Individual lecture records	Unique video_id per playlist
progress	id, user_id, lecture_id, completed_at	Completion tracking records	Unique (user, lecture) pair

The UNIQUE constraint on progress(user_id, lecture_id) is the primary guard against duplicate completion records. The lectures table similarly uses a UNIQUE constraint on (playlist_id, video_id) to prevent a playlist import from inserting the same YouTube video twice. Together, these constraints implement the data integrity feature described in the project requirements.

7. Real-Life Use Cases

The Study Tracker is designed to serve a broad range of learners who consume educational video content. The following use cases illustrate real-world application scenarios:

7.1 Competitive Exam Aspirants

Students preparing for competitive exams such as GATE, JEE, UPSC, or CAT frequently follow curated YouTube playlists covering specific subjects. The Study Tracker allows them to import playlists for each subject (Mathematics, Reasoning, Subject Knowledge), track daily progress, and use the estimated completion date to align their study plan with their exam schedule.

7.2 Self-Learners in Technology

Individuals learning programming, web development, data science, or design through YouTube tutorials can use the app to ensure they complete every video in a course playlist without losing track. The multi-subject feature lets them manage several courses concurrently — for example, tracking a Python course, a SQL course, and a Machine Learning course simultaneously.

7.3 Coaching Institutes

Educational institutes that record and upload lecture content as YouTube playlists can deploy the Study Tracker to monitor batch-level progress. Teachers can create subject entries corresponding to each batch's playlist and review aggregate completion data to identify under-performing students or topics that need re-teaching.

7.4 Teachers & Course Creators

Educators who assign YouTube playlist content as homework or supplementary material can use the app to verify completion. Students submit their progress data, and teachers review subject-level completion rates, making it easy to follow up with students who have fallen behind.

8. Future Improvements & Roadmap

The current implementation delivers a solid foundation of features. The following planned enhancements represent the next evolution of the platform:

Feature	Description
AI Study Planner	Automatically generate a daily study schedule by analyzing remaining lectures, subject difficulty, and the student's available hours, powered by an AI/ML recommendation engine.
Streak & Gamification System	Introduce a daily streak counter that rewards consistent study habits. Badges, milestones, and leaderboards would make learning more engaging and motivating.
Smart Adaptive Scheduling	Detect weak subjects based on low completion velocity and automatically increase their priority in the generated study plan, ensuring balanced subject coverage.
Multi-Platform Import	Extend import support beyond YouTube to include Udemy, Coursera, LinkedIn Learning, and custom URLs, making the app a universal study tracker.
Mobile Application	Develop native iOS and Android applications using React Native, sharing the same FastAPI backend, to allow students to mark lectures complete from their phones.
Social Study Rooms	Enable group study tracking where friends can share progress, compare completion rates, and set joint goals, introducing a collaborative element to self-study.

9. Challenges & Learnings

9.1 Technical Challenges

- YouTube API Pagination: Playlists with more than 50 videos require multiple paginated API calls. Handling nextPageToken correctly to ensure all lectures are fetched was a key implementation challenge.
- Duplicate Prevention: Designing idempotent endpoints that gracefully handle re-submission without returning errors required careful use of database constraints and HTTP status codes (200 vs 201 vs 409).
- Progress Calculation Performance: Initially calculated in Python application code using loops, progress calculation was later moved into a single SQL aggregation query for better scalability.
- CORS and API Key Security: Keeping the YouTube API key server-side (in the FastAPI backend) rather than exposing it in the React frontend required refactoring the initial architecture.

9.2 Key Learnings

- Layered architecture (route → service → repository) significantly improves testability and makes future feature additions straightforward.
- Database constraints should be the last line of defense for data integrity, complementing application-level validation.
- FastAPI's dependency injection system is powerful for managing shared resources like database sessions and API clients across routes.
- Building with a clear API contract (OpenAPI schema) from the beginning ensures the frontend and backend teams can work in parallel without blocking each other.

10. Conclusion

The Study Tracker Web App successfully addresses a genuine gap in the self-learning ecosystem. By combining automated YouTube playlist import, lecture-level completion tracking, and intelligent analytics, it transforms the fragmented experience of following online video courses into a structured, measurable learning journey.

The technical architecture — React on the frontend, FastAPI on the backend, and MySQL as the data store — is both modern and pragmatic. The layered design ensures that each component of the system can be extended, tested, or replaced independently. The YouTube Data API integration demonstrates how third-party services can be leveraged to automate data entry that would otherwise be manual and error-prone.

Looking ahead, the planned enhancements — AI study planning, streak gamification, and multi-platform import — position the Study Tracker as a comprehensive learning companion rather than a niche tool. With a strong foundation in place, the platform is well-suited for expansion into a fully-featured edtech product.

> Summary of Achievements:
> - ✓ Fully functional YouTube playlist import with pagination support
> - ✓ Real-time progress tracking with duplicate-safe updates
> - ✓ Subject-wise analytics dashboard with time-series charts
> - ✓ Completion prediction engine based on study velocity
> - ✓ Clean REST API with auto-generated OpenAPI documentation
> - ✓ Scalable layered architecture ready for future enhancements

End of Report